

AHSANULLAH UNIVERSITY OF SCIENCE AND TECHNOLOGY (AUST)
141 & 142, Love Road, Tejgaon Industrial Area, Dhaka-1208.



Department of Computer Science and Engineering
Program: Bachelor of Science in Computer Science and Engineering

Project Final Report

Course No: CSE-1200

Course Title: Software Development – I

Project Title: Prince of Persia: Escape from the Palace

Date of Submission: 30/09/26

Submitted to

Saha Reno

Assistant Professor, Department of CSE, AUST.

Md. Zahid Hossain

Lecturer, Department of CSE, AUST.

Submitted by,

Name: Makdad Ibn Mannan

Student ID: 00725105101153

Name: Adnan Mukid

Student ID: 00725105101159

Name: Sudipta Roy

Student ID: 00725105101170

Name: Lithun Zira Iyesmin Prova

Student ID:00725105101156

Introduction

1.1 Project Overview

Prince of Persia: Escape from the Palace is a 2D side-scrolling action-adventure platform game developed in C/C++ using the iGraphics library. The player controls the Prince and must escape through three levels containing running, jumping, climbing, platforming, sword combat, and different environmental hazards such as bats, poison water, floods, and piranhas. The first two levels are completed by defeating all guards, while the final level is completed by surviving the escape sequence and reaching the exit door. The game also records the player's fastest completion time for each level. It runs on Windows at a fixed resolution of 1350 × 680 pixels.

1.2 Objective

- To develop a complete 2D action-platform game using C/C++ and iGraphics.
- To implement player movement, jumping, climbing, gravity, and fall damage.
- To implement sword combat and enemy behaviour.
- To develop a health system with health pickups, checkpoints, and game-over conditions.
- To create multiple levels with unique environments, artwork, and hazards.
- To implement different game screens such as splash, home, story, instructions, credits, game over, level complete, and victory.
- To add background music and sound effects using the Windows Multimedia API.
- To implement a level timer, progress bar, and persistent best-record system.

Game Development

2.1 Tools & Technologies Used

- **Programming Language:** C, C++
- **IDE:** Visual Studio 2010/2013
- **Graphics Library:** iGraphics
- **Image loading:** stb_images.h (136 image assets)
- **Audio:** Windows Multimedia API (winmm)
- **Operating system:** Windows PC
- **AI assistance:** Claude

2.2 Core Features

Player Movement

The Prince can run using A/D or the arrow keys, jump across gaps, and climb higher ledges. Gravity and fall damage are applied when the player leaves a platform.

Combat

The player can attack using the Space key, with sword attacks shown through animated swing sequences. Enemies have their own health, movement, and attack behaviour.

Health and Checkpoints

The player has a maximum health of 350, and collectible hearts restore health. Switches work as checkpoints and record the player's progress. The game ends when the player's health reaches zero.

Level 2 – The Caves

Bats attack the player, and the Prince can crouch to avoid them. Poison water causes a fatal drowning sequence.

Level 3 – Final Escape

Collapsing platforms and rising flood water create environmental hazards. The player can swim and climb out of the water, while piranhas attack underwater. A Guardian boss appears before the final escape, and the level ends when the Prince reaches the exit door.

Timer, Progress and Records

A high-resolution timer records actual gameplay time, while a progress bar shows the player's physical progress through the level. The fastest completion time for each level is permanently stored.

Audio

Background music plays during gameplay and on the victory screen. Running, button-click, and attack sounds are implemented, while some additional sound effects are supported but currently unavailable.

Implementation Details

3.1 Code Structure

The project has **one translation unit**. `main.cpp` is the only `.cpp` file compiled; every other part of the game is a header that `main.cpp` includes. The headers are therefore an organisational device rather than separately compiled modules,

File	Responsibility
<code>main.cpp</code>	Starts the game and initializes the main systems.
<code>iGraphics.h</code>	Handles graphics and the game window.
<code>stb_image.h</code>	Loads PNG images.
<code>common.h</code>	Contains common libraries and Windows headers.
<code>game_types.h</code>	Defines the main game structures.
<code>game_globals.h</code>	Stores constants, settings, and game states.
<code>next_level.h</code>	Creates the three levels, enemies, and hazards.
<code>controls.h</code>	Handles controls, movement, physics, and enemy AI.
<code>combat.h</code>	Handles attacks and damage.

File	Responsibility
images.h	Draws the game and different screens.
records.h	Manages the timer, progress, and records
sounds.h	Handles music and sound effects.
home.h	Handles the home screen and buttons.
reset.h	Starts and restarts levels.

Screenshots

All figures below are captured from the running game.

Figure 1: Home Screen



The main menu with its five options — START GAME, STORY, INSTRUCTIONS, CREDITS and RECORDS.

Figure 2: Gameplay (Level 1)



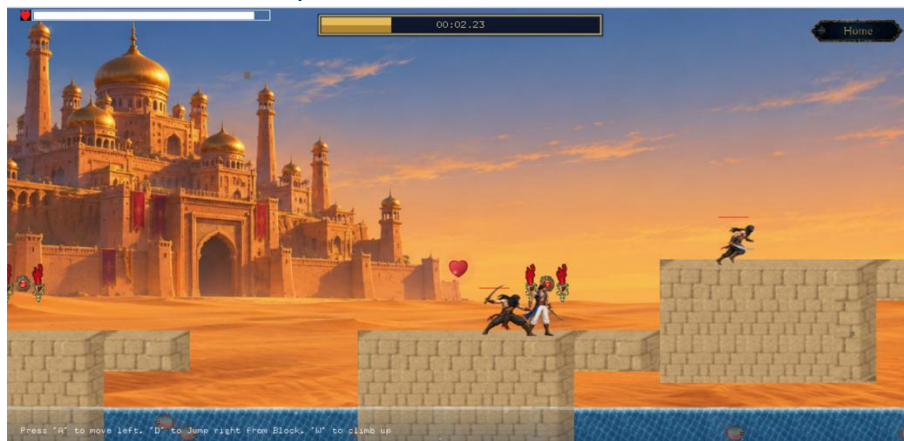
Level 1 inside the palace. Visible are the health bar (top-left), the level progress bar with the elapsed time inside it (top-centre), the in-game HOME button (top-right), and the checkpoint confirmation banner shown after operating a switch.

Figure 3: Level 2 — The Caves



The second level, with its own cave tile set and backdrop, the poison water hazard, and the collectible health hearts.

Figure 4: Level 3 — The Final Escape



The third level outside the palace, with the sand tile set, the rising flood along the bottom of the screen containing the piranhas, and a health heart above the platforms.

4. GitHub Collaboration

4.1 Repository Management

The project was managed using GitHub to enable efficient collaboration, version control, and code sharing among team members. A dedicated repository was created where all source code, assets, and documentation were maintained.

4.2 Contribution Workflow

The team followed a structured workflow:

1. Repository initialized with project structure
2. Members cloned the repository to their local systems
3. Features were developed in separate branches

4. Changes were committed with meaningful messages
5. Pull requests were created and reviewed
6. Approved changes were merged into the main branch

4.3 Individual Contributions

Each member was responsible for specific modules:

- **Member 1: Makdad Ibn Mannan:** Combat & enemies
- **Member 2: Adnan Mukid:** Player movement, controls & physics
- **Member 3: Sudipta Roy:** Levels, platforms, obstacles
- **Member 4: Lithunzira Iyesmin Prova:** UI, graphics & sound

Challenges & Limitations

5.1 Challenges

Challenge	Solution
The game crashed because multiple MCI audio devices were opened	MP3 files were decoded once and played using
Players received fall damage immediately after restarting.	A flag was added to ignore the first landing damage.
The checkpoint message background was too small.	its size was adjusted automatically according to the text.
The timer and progress bar did not work correctly.	Gameplay time and physical player progress were calculated separately.
Piranhas could move outside the water or cluster together.	Their movement was restricted and a separation system was added.
Returning to Level 1 caused collision problems.	Level 1 was rebuilt whenever the game was restarted.

5.2 Limitations

- **Single player only.** There is no second player and no multiplayer mode.
- **Three levels.** The game ends after Level 3; there is no level beyond the escape sequence.
- **Limited enemy variety.** The roster is palace guards, Level 2's bats, Level 3's piranhas and a single Guardian boss. All the guards share one behaviour and one sprite set.
- **No save or resume.** Only the best times persist. There is no way to save a game in progress and continue it later; closing the game loses the current run.
- **Fixed window size.** The window is fixed at 1350 × 680 and is not resizable; there is no full-screen mode.

Conclusion & Future Improvements

6.1 Summary

The project successfully delivers a complete and playable 2D action-platform game using C/C++ and the iGraphics library. The game features three levels, each with unique environments, enemies, hazards, and challenges.

The game also includes different screens, background music, sound effects, a timer, a progress bar, and a record system. Overall, this project helped us learn how to use graphics, game controls, collision detection, game logic, and audio to build a complete game.

6.2 Future Enhancements

The following are **not currently implemented** and are proposed as future work.

- **Additional levels** beyond the three that exist, reusing the existing level-building pattern.
- **More enemy types** with distinct behaviour — ranged attackers, or guards that patrol between platforms rather than staying on one.
- **Expanded combat** — blocking, parrying, combos, or a second weapon.
- **A save/resume system** so the game can be continued later.
- **Difficulty modes** — the game currently has none; selectable difficulty could scale enemy health, damage and the Level 3 flood rate.